Steering Agent Behavior via a Domain Expert-Driven Alignment-to-Optimization Bridge

Wesley Pasfield
University of San Diego
San Diego, California, USA
wpasfield@sandiego.edu

Abstract

Aligning compound AI agents with domain expertise typically requires manual prompt engineering or scorer design that drifts from actual expert quality criteria. We present At-Bat Alignment Bridge, a system that makes agent behavior steerable: the only manual step is for domain experts to label traces. From those labels, an automated bridge produces a calibrated evaluation judge (via MemAlign), an optimized system prompt (via GEPA), and composable agent skills (via GEPA's `optimize_anything`). Because the judge is calibrated to expert feedback before optimization begins, every downstream change reflects the expert's definition of quality, and all artifacts are versioned in MLflow for auditability. We demonstrate the bridge using a baseball hitting-analysis assistant with graph-enforced tool routing, per-thread conversation memory, and parallel tool execution. The graph makes deterministic tool use inspectable before probabilistic fallback, while parallel execution keeps the live demo responsive. In this example, the bridge culminates in an agent that outperforms the original by 15.7% as evaluated by the aligned judge.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; • **Information systems** → *Data management systems*; • **Software and its engineering** → *Software creation and management*.

Keywords

Compound AI systems, agent evaluation, judge alignment, prompt optimization, expert-steered optimization, tool-augmented agents

ACM Reference Format:
Wesley Pasfield. 2026. Steering Agent Behavior via a Domain Expert-Driven Alignment-to-Optimization Bridge. In *ACM Conference on AI and Agentic Systems (CAIS '26), May 26–29, 2026, San Jose, CA, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3786335.3813200

1 Introduction

Compound AI systems compose multiple components, including large language models (LLMs), tools, retrievers, and evaluators, in pipelines that are optimized and executed at inference time [22]. A central challenge is aligning agent behavior with domain expertise: closing the loop between subject-matter experts (SMEs), who define quality, and the engineering artifacts (prompts, tool schemas, evaluation judges) that govern the agent. Existing frameworks address pieces of this challenge but not the whole. DSPy/GEPA [1, 8] optimizes prompts but assumes a scorer already reflects domain quality; observability platforms such as LangSmith [10] support trace annotation but do not calibrate judges from labels; and automated prompt search methods (APE, OPRO [20, 24]) treat the scorer as fixed despite known LLM-as-judge bias [23]. Benchmarks and agent-learning methods (e.g., AgentBench, Reflexion [13, 18]) likewise do not provide a closed loop from expert labels to judge calibration to prompt optimization and skill generation.

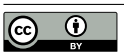
This work demonstrates At-Bat Alignment Bridge, a system in which that connection is automated and governed. The loop proceeds in five steps: an LLM-as-a-judge evaluates selected traces; domain experts label those same traces via an MLflow labeling session; MemAlign [14] calibrates an LLM judge to the domain-expert feedback; GEPA [1] (Genetic-Pareto prompt optimization) updates the system prompt using the aligned judge as the scorer; and GEPA's `optimize_anything` synthesizes the optimized prompt, aligned-judge guidelines, tool signatures, and evaluated traces into composable agent skill modules loaded at runtime. All artifacts, including traces, datasets, judges, prompts, and skills, are versioned in MLflow and bound to a single experiment. Developers can run the loop end-to-end or insert manual gates (e.g., requiring approval before prompt promotion or judge registration).

The core claim is that a domain expert can steer agent behavior toward their own quality criteria through labeling alone, without requiring manual prompt editing (Section 3.1). The two primary contributions are: (1) an alignment-to-optimization bridge that converts expert trace labels into a calibrated judge, an optimized prompt, and composable agent skill modules, with labeling as the only manual step (Section 3.1); and (2) eval-driven agent skill generation via GEPA's `optimize_anything` [1, 7] that iteratively refines tool signatures, the optimized prompt, and aligned-judge guidelines into modular agent skill files loaded at runtime (Section 3.4). Two supporting design decisions make the bridge demonstrable in an interactive setting: graph-enforced tool routing keeps deterministic evidence visible before probabilistic fallback (Section 3.2), and parallel tool execution via per-thread MCP clients keeps multi-tool turns fast enough for live use (Section 3.3). We demonstrate these through an assistant for baseball hitting analysis.

2 System Architecture

2.1 Demonstration Domain and Data

The demonstration domain is baseball hitting analysis because it combines structured statistics, ambiguous tactical questions, and terminology that makes generic scoring brittle. The data layer uses



CAIS '26, San Jose, CA, USA

ACM ISBN 979-8-4007-2415-2/2026/05
https://doi.org/10.1145/3786335.3813200

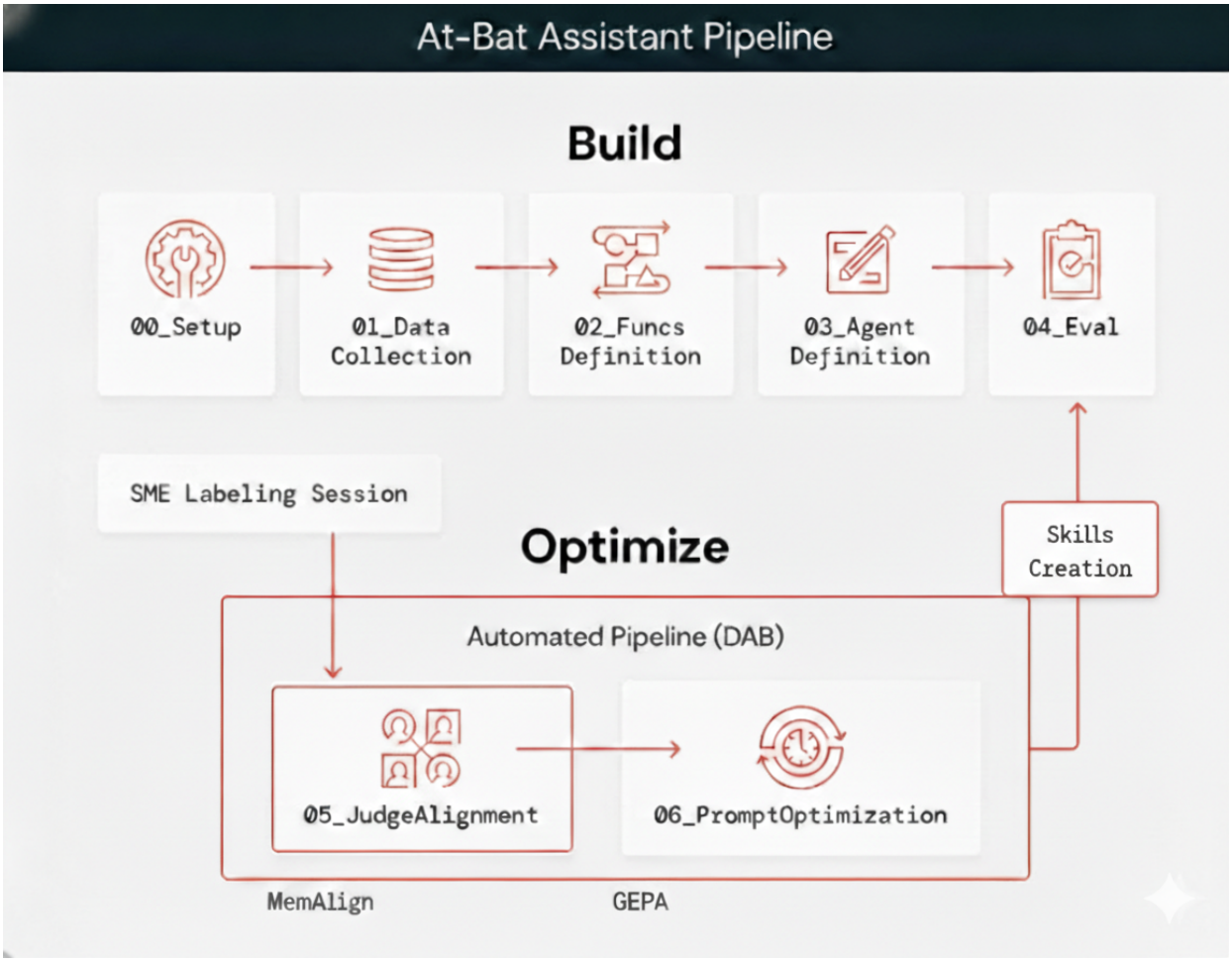


Figure 1: Pipeline overview. Build phase: Setup, Data Collection, Function Definition, Agent Definition, Evaluation. Optimize phase: SME Labeling Session feeds the Automated Pipeline (Judge Alignment, Prompt Optimization), which loops back to Evaluation.

pitch-level Statcast tracking data [15], including pitch type, velocity, movement, count, handedness, and batted-ball measurements. These records are transformed into query-ready feature tables and vector search indices for player similarity. Evaluation and optimization questions are synthetic, not scraped from analyst logs. That choice keeps the artifact reproducible, but limits claims about real analyst query distributions.

2.2 Pipeline Stages

The pipeline comprises seven stages driven by a shared JSON configuration.

Data collection. The Statcast records are processed into query-ready pitcher and batter feature tables in Delta, with MinMax-scaled embedding vectors indexed for similarity queries [4].

Function definition. Each capability is exposed as a schema-governed function with typed parameters and coverage docstrings (e.g., count tendencies, matchup history, similarity search), enabling semantic tool selection instead of hard-coded routing.

Agent definition. The agent is implemented as an MLflow ResponsesAgent [16] backed by a LangGraph [9] state machine. It loads the system prompt from a versioned prompt registry, invokes a single LLM with tool definitions, and executes tools in parallel when possible (Section 3.3). Conversation state is persisted per-thread in a managed PostgreSQL instance (Lakebase [5]) that serves as the LangGraph checkpoint store, supporting multi-turn interaction and horizontal scaling of the serving endpoint.

Evaluation. Evaluation and optimization datasets are generated synthetically via LLM API calls. The initial evaluation uses MLflow GenAI's `RelevanceToQuery` scorer, a custom `Guidelines` scorer for baseball-specific language, and a custom 1 to 5 judge over tool usage, factual accuracy, and actionability (Appendix C); traces are tagged and merged into a versioned dataset for labeling and alignment. The held-out comparison in Section 3.1 reports the aligned judge score only, so all configurations are compared under the same expert-calibrated criterion.

Judge alignment. Out-of-the-box judge scores may diverge from human assessments. MemAlign [14] aligns the custom 1 to 5 baseball analysis judge to expert labels via a dual-memory system that distills feedback into reusable principles and edge-case examples (Section 3.1).

Prompt optimization. The aligned judge scores an optimization dataset via GEPA [1]. The best prompt is promoted to production (Section 3.1).

Agent skill generation. As a final pipeline stage, the system synthesizes tool signatures, the optimized prompt, the aligned judge's learned guidelines, and traces containing both LLM and domain expert feedback into modular agent skill files. These files decompose the agent's capabilities into self-contained, reusable modules that are loaded at runtime (Section 3.4).

2.3 Orchestration and Artifact Management

A single MLflow experiment holds runs, traces, and registered judges. Base and aligned judges are registered by name; the Prompt Registry versions prompts and exposes a production alias consumed by the serving endpoint. Developers can automate the full cycle or insert gates (e.g., requiring human approval before alias promotion or judge registration), balancing automation with governance.

3 Technical Contributions

3.1 The Alignment-to-Optimization Bridge

The central mechanism of the expert-steered optimization loop is the alignment-to-optimization bridge: human feedback calibrates the evaluation judge, and the aligned judge drives automated prompt optimization. Without this bridge, optimization targets a scorer that may diverge from expert quality criteria, improving a metric that does not reflect actual quality. The base judge is used to bootstrap trace scoring and create the labeling set. After the labeling session, the aligned judge replaces it as the optimization and held-out evaluation signal.

Judge alignment via MemAlign. Domain experts review agent traces and provide scores (in our case, 1 to 5 Likert ratings of tool usage appropriateness, factual accuracy, and tactical quality). MemAlign [14] aligns the judge to these human assessments through a dual-memory architecture inspired by human cognition. Semantic memory distills expert feedback into generalizable principles (e.g., "always penalize recommendations that lack tactical specificity"). Episodic memory retains specific edge-case examples where the judge and human disagreed. At inference time, MemAlign constructs a working memory by combining all stored principles with the most relevant retrieved examples, enabling the judge to score new traces informed by accumulated expert knowledge. In this run, 30 labels produced seven semantic guidelines and 30 episodic examples (Appendix C). Because MemAlign learns from natural-language feedback rather than requiring large labeled datasets, it continues to improve as feedback accumulates, a property termed memory scaling [14].

Prompt optimization via GEPA. The aligned judge is passed as GEPA's scorer [1]. GEPA reflects over full trajectories (tool calls, tool outputs, final responses), proposes prompt updates, and keeps high-performing variants on a Pareto frontier. We run five independent optimizations on disjoint 20-example subsets from a 100-question pool, each with a budget of 100 scorer calls. This setup tests whether a small expert-labeling investment can still drive consistent gains (Appendix D).

Compute cost. Prompt optimization dominated cost in the reported run. Each GEPA run uses 100 scorer calls on a 20-example subset, and the five-run sweep therefore uses 500 aligned-judge scorer calls plus agent rollouts for the evaluated candidates. MemAlign alignment runs once over 30 labeled traces and produces reusable judge memory. Skill generation uses text-only scoring and reflection over candidate skill files, so it avoids live agent rollouts during refinement; in practice this stage converged in minutes without a serving-endpoint evaluation loop.

Why the bridge matters. Neither component is sufficient alone. Alignment without optimization improves evaluation fidelity but does not improve the agent. Optimization without alignment risks reward hacking: the prompt learns to satisfy a mis-calibrated scorer while potentially degrading actual quality. Notably, alignment may lower the baseline score relative to an uncalibrated judge, because the calibrated judge enforces criteria the base judge overlooked. A lower but accurate starting point means that subsequent optimization gains reflect actual quality improvements rather than exploitation of scorer blind spots.

The bridge produces a calibrated judge, an optimized prompt, and agent skill modules (Section 3.4), with domain expert labeling as the only manual step (Appendix A). The same labels also calibrate an evaluation suite (aligned judge, per-example scores, learned guidelines) that developers can inspect for targeted manual fixes. The bridge's role is to stretch expert feedback as far as possible through automation, producing immediate gains that complement manual improvement.

To measure robustness, the experiment proceeds in two phases. In Phase A, GEPA optimizes the base agent's prompt across five independent runs on disjoint 20-question subsets drawn from a 100-question synthetic pool, each with a budget of 100 scorer calls. In Phase B, the best optimized prompt from Phase A is used to evaluate both the base agent and the skills-enhanced agent on a held-out evaluation dataset (the 30-question set from the initial evaluation stage, unseen during optimization). Both agents use the identical optimized prompt, so the only variable is the presence of agent skills. This design isolates the incremental contribution of skills from the gains already achieved by prompt optimization alone, while also testing whether the optimized prompt generalizes beyond the training data. All runs use the same aligned judge.

Table 1 summarizes the results. Across five Phase A runs, GEPA consistently improved the optimization-set checkpoint from 2.95/5 to 3.23 ± 0.15/5 (+9.5%). On held-out data (Phase B), the original agent scored 3.00/5; with the optimized prompt this rose to 3.33/5, and with skills to 3.47/5 (+4.2% over optimized prompt, +15.7% over held-out baseline). Because the held-out comparison uses the same optimized prompt for base and skills agents, the difference is attributable to skills. This supports the claim that the bridge enables controllable behavior change through expert feedback, while keeping all artifacts (judges, prompts, scores, traces) auditable in MLflow.

Evaluation scope. The system's author served as the proxy domain expert for this demonstration, and the aligned judge was calibrated from 30 labels. This creates a circularity risk: the same

Table 1: Aligned-judge scores by evaluation setting. Phase A reports GEPA optimization checkpoints (5 runs on disjoint 20-question subsets, 100 scorer calls each). Phase B reports held-out performance (30 questions unseen during optimization). In Phase B the optimized and optimized+skills agents share the same prompt, isolating the effect of skills. All rows use the same 30 expert labels for judge calibration.

Stage	Score (1 to 5)	Gain	n
Phase A: Optimization set (GEPA checkpoints)			
Baseline (pre-opt)	2.95 ± 0.19	–	5
Align + GEPA	3.23 ± 0.15	+9.5%	5
Phase B: Held-out set (same aligned judge)			
Original agent	3.00	–	30
Optimized prompt	3.33	+11.0%	30
+ Skills	3.47	+15.7%	30

person's criteria shape the judge and the reported held-out score. The experiment mitigates this only partially by using held-out questions unseen during prompt optimization and by isolating prompt and skill effects under the same judge. The claim is therefore about the bridge's mechanism, whether labeling alone can drive automated optimization, with no claim of universal baseball expertise. A different expert would produce a different judge, prompt, and agent skill set, while the bridge would function identically. Stronger validation would use multiple professional hitting coaches, report inter-annotator agreement, and measure the label-efficiency curve across larger labeling budgets.

3.2 UC Function and Genie Prioritization

The agent is equipped with two classes of tools. Unity Catalog (UC) functions [6] are typed SQL functions that provide deterministic answers; vector search indices complement them for similarity queries. Genie is a natural-language data-exploration interface that supports probabilistic answers when the query is outside the scope of the UC functions. Tool-augmented agents such as ReAct [21] and Toolformer [17] interleave reasoning and action steps but delegate all routing decisions to the LLM, whether via prompting or learned API calls.

Our system instead enforces a graph-level two-phase pattern in LangGraph. In the first phase, each UC function carries a detailed docstring describing its coverage. The system prompt instructs the LLM to prefer UC functions to prioritize determinism, and UC functions may be invoked in parallel (Section 3.3). After UC results are returned, a dedicated sufficiency evaluation node categorizes the response as `FULLY_ANSWERED`, `PARTIALLY_ANSWERED`, or `NOT_ANSWERED`. A conditional edge routes: if sufficient, the graph proceeds to synthesis; if not, a Genie fallback node is invoked with only the unanswered parts, and its output is combined with UC results in the synthesis node. This routing matters for the alignment loop because the traces expose which evidence came from deterministic functions and which came from fallback data exploration. In the demonstration artifact, the streaming chat interface renders each tool call inline, and UC function results are visually distinguished from Genie fallback results, making the deterministic-probabilistic balance directly observable to the user.

3.3 Parallel Tool Calling

When the LLM requests multiple tool calls in one turn, sequential execution causes latency to grow linearly. The Databricks SDK's workspace client serializes requests when shared across threads, so the implementation creates a fresh workspace client and MCP client [3] per tool-calling thread, using credentials cached at module load. A ThreadPoolExecutor dispatches each call to an isolated client; results are collected via `as_completed` and keyed by tool-call ID. Per-thread client instantiation adds 10 to 30 ms overhead, negligible compared to the network round-trip saved by parallel execution. Latency therefore scales as the single slowest call rather than the sum; in the evaluated agent, queries triggering four tool calls completed in roughly 2 to 2.5 s versus approximately 8 s sequentially.

3.4 Eval-Driven Agent Skill Generation and Runtime Integration

Modular agent skill definitions decompose the agent's capabilities into self-contained, file-based modules generated from empirical artifacts rather than manual authoring, similar in spirit to Voyager's [19] experience-driven skill library but grounded in human expert feedback. Recent benchmarking finds that agent self-generated skills provide negligible benefit (-1.3pp on average) because agents produce imprecise procedures [12]; our approach aims to sidestep this by grounding generation in tool signatures, the optimized prompt, the aligned judge's memory, and evaluated traces with expert feedback.

Skill generation uses GEPA's `optimize_anything` [1, 7], which treats skill text as an optimizable parameter. The optimizer follows GEPA's three-stage loop: candidate skill sets are scored against a composite evaluator measuring tool coverage, structural completeness, and example richness; a reflection step analyzes scorer diagnostics to identify why specific skills scored poorly (e.g., missing fallback logic for absent data, incomplete parameter constraints); and a curation step generates improved candidates informed by those diagnostics. This loop operates on skill text alone, requiring no agent re-execution, so cost is bounded by LLM inference calls to the reflector and scorer rather than by end-to-end agent rollouts. The distinction matters in practice: each agent rollout invokes multiple tool calls against live data services, while skill-text refinement is a closed-loop text optimization that converges in minutes.

Each skill follows the Agent Skills architecture [2]: YAML frontmatter (name, description) for lightweight discovery and a structured body decomposed into separate workflow, gotcha, and example files (Appendix G). This produced seven skills for the demonstration agent. Skills are written to a Unity Catalog Volume as the canonical store and persisted to Lakebase for runtime access. To make the causal chain explicit: the aligned judge is the direct optimization signal for prompt search (Section 3.1), while skill generation consumes aligned-judge artifacts (guidelines and examples) as inputs and uses a structural skill-quality scorer during `optimize_anything`. The effect of those generated skills is then measured downstream with held-out aligned-judge evaluation in

Table 1. This is an end-to-end behavioral evaluation, not an independent audit of skill text quality. A stronger follow-up would compare generated skills against manually authored skills, ask experts to rate skill correctness and usability, and ablate workflow, gotcha, and example sections separately.

3.5 Runtime Integration via Progressive Disclosure

At startup, agent skill metadata (name and description) is loaded from the Lakebase `skills` table and appended to the system prompt; full agent skill content is loaded on demand through a `load_skill` tool [11]. The tool execution node enforces skill-first prioritization: when the LLM requests both `load_skill` and UC function calls in one turn, only the agent skill calls execute, so the agent skill instructions are read before selecting functions. Genie-related agent skills follow a separate path: they are excluded from the system prompt and instead loaded dynamically in the Genie fallback node, where their guidance is provided to the fallback query. Because agent skill generation is driven by the same feedback loop, re-running the generator after additional labeling rounds produces updated agent skills without manual authoring or redeployment.

4 Demonstration Plan

This demonstration targets both novel technical contribution and compelling interactive experience. It has two components. The At-Bat Assistant is a Databricks App exposing the agent through a streaming chat interface where each tool call, UC/Genie routing decision, and skill load is rendered inline. Representative queries highlight parallel tool execution and pre- vs. post-optimization quality differences. The second component walks through At-Bat Alignment Bridge, starting with a live MLflow labeling session, moving through MemAlign judge calibration, and showing GEPA prompt optimization via precomputed results (the genetic search requires approximately 100 scorer calls per run). The demo follows how labels become judge memory, prompt changes, and generated skills. A video covers the end-to-end loop.[1] Source code and notebooks are available in the accompanying repository.[2] The Databricks App is deployed on private infrastructure; reviewer access can be arranged upon request.

5 Conclusion

A single labeling session by a domain expert can drive three automated changes: a calibrated judge, an optimized prompt, and composable agent skills. The alignment-to-optimization bridge makes steerability operational by ensuring optimization follows expert-induced criteria rather than a fixed generic scorer. In our demonstration, optimization checkpoints improve from 2.95/5 to 3.23/5 across five GEPA runs, and held-out performance progresses from a 3.00/5 baseline for the original agent to 3.47/5 for optimized+skills behavior under the same aligned judge. Because expert time is the bottleneck, the bridge is designed to maximize its impact: one labeling session can be amplified into auditable, iterative agent updates without manual prompt or skill editing. While the demonstration uses Databricks infrastructure, MLflow [16], MemAlign, and GEPA operate on traces and scores rather than platform APIs, making the bridge portable. The main open questions are empirical: how the judge scales with 100, 500, or 1000 labels; whether independent experts converge or induce meaningfully different agents; whether generated skills transfer to related queries; and how this pattern behaves outside baseball.

[1] https://vimeo.com/1171406108
[2] https://github.com/WesleyPasfield/at-bat-assistant

References

[1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457 [cs.CL]

[2] Anthropic. 2025. Agent Skills: Modular Capabilities for AI Agents. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview Accessed: 2026-02-09.

[3] Databricks. 2025. Databricks MCP Server. https://docs.databricks.com/en/machine-learning/mcp.html Accessed: 2026-02-06.

[4] Databricks. 2025. Databricks Vector Search Documentation. https://docs.databricks.com/en/generative-ai/vector-search.html Accessed: 2026-02-08.

[5] Databricks. 2025. Lakebase: Managed PostgreSQL on Databricks. https://docs.databricks.com/en/lakebase/index.html Accessed: 2026-02-06.

[6] Databricks. 2025. Unity Catalog Documentation. https://docs.databricks.com/en/data-governance/unity-catalog/index.html Accessed: 2026-02-06.

[7] GEPA Team. 2026. optimize_anything: A Universal API for Optimizing any Text Parameter. GEPA project blog. https://gepa-ai.github.io/gepa/blog/2026/02/18/introducing-optimize-anything/ Accessed: 2026-02-26.

[8] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=sY5N0zY5Od

[9] LangChain. 2024. LangGraph: Build Stateful, Multi-Actor Applications with LLMs. https://langchain-ai.github.io/langgraph/ Accessed: 2026-02-06.

[10] LangChain. 2025. LangSmith: Observability and Evaluation for LLM Applications. https://docs.smith.langchain.com/ Accessed: 2026-02-09.

[11] LangChain. 2025. Skills: Multi-Agent Patterns. https://docs.langchain.com/oss/python/langchain/multi-agent/skills Accessed: 2026-02-10.

[12] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670 [cs.AI]

[13] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=zAdUB0aCTQ

[14] Veronica Lyu, Kartik Sreenivasan, Samraj Moorjani, Alkis Polyzotis, Sam Havens, Michael Carbin, Michael Bendersky, Matei Zaharia, and Xing Chen. 2026. MemAlign: Building Better LLM Judges From Human Feedback With Scalable Memory. Databricks Mosaic Research Blog. https://www.databricks.com/blog/memalign-building-better-llm-judges-human-feedback-scalable-memory Accessed: 2026-02-09.

[15] MLB Advanced Media. 2025. Statcast Search, Baseball Savant. https://baseballsavant.mlb.com/statcast_search Pitch-level tracking data including velocity, spin rate, movement, and batted-ball metrics. Accessed: 2026-02-08.

[16] MLflow Authors. 2025. MLflow GenAI Documentation. https://mlflow.org/docs/latest/genai/index.html Accessed: 2026-02-06.

[17] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://openreview.net/forum?id=Yacmpz84TH

[18] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://openreview.net/forum?id=vAElhFcKW6

[19] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied

Agent with Large Language Models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://openreview.net/forum?id=ehfRiF0R3a
[20] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=Bb4VGOWELI
[21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X
[22] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. 2024. The Shift from Models to Compound AI Systems. Berkeley AI Research Blog. https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/ Accessed: 2026-02-06.
[23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://openreview.net/forum?id=uccHPGDlao
[24] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large Language Models Are Human-Level Prompt Engineers. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=92gvk82DE-

A Alignment-to-Optimization Bridge Algorithm

Algorithm 1 summarizes the full loop from domain expert labels through judge alignment, prompt optimization, and agent skill generation. Steps 1 to 3 consume the same 30 expert labels; Step 3 uses GEPA's `optimize_anything` for iterative skill refinement; Step 4 feeds the next cycle.

Listing 1: Alignment-to-optimization bridge.

```
# INPUTS
traces = evaluate(agent, eval_dataset, scorers)
labels = labeling_session(traces, domain_expert)

# STEP 1: Judge alignment
aligned_judge = base_judge.align(
    traces=labels,
    optimizer=MemAlignOptimizer()
)
register(aligned_judge)  # MLflow scorer

# STEP 2: Prompt optimization
best_prompt = optimize_prompt(
    agent=agent,
    dataset=optimization_dataset,
    scorer=aligned_judge,  # calibrated signal
    optimizer=GEPAOptimizer()
)
register(best_prompt)  # Prompt Registry
promote(best_prompt, alias="production")

# STEP 3: Agent skill generation via optimize_anything
sources = {
    "tools": load_tool_signatures(),
    "prompt": load_prompt("production"),
    "judge": aligned_judge.memory,
    "traces": labeled_traces_with_feedback
}
skills = optimize_anything(
    objective=sources,
    scorer=skill_quality_scorer,
    optimizer=GEPAOptimizer()
)
write_to_volume(skills)    # UC Volume
write_to_lakebase(skills)  # runtime access

# STEP 4: Re-evaluate (optional next cycle)
updated_agent = reload(agent, best_prompt, skills)
new_traces = evaluate(
    updated_agent,
    eval_dataset,
    scorers=[aligned_judge]
)
```

B Artifact-to-Claim Traceability

Table 2 maps each primary paper claim to the concrete implementation artifacts used in the demonstration repository.

C Base Judge vs. Aligned Judge

The base judge uses a generic five-point rubric:

> Evaluate if the response appropriately analyzes the available data and provides an actionable recommendation.

(1) Incorrect data interpretation or no recommendations.
(2) Irrelevant feedback or weak recommendations.
(3) Relevant feedback with some strategic advantage.
(4) Relevant feedback with strong strategic advantage.
(5) Relevant feedback with excellent strategic advantage.

After MemAlign alignment on 30 expert-labeled traces, the judge acquires seven domain-specific guidelines (semantic memory) and 30 scored examples (episodic memory). The learned guidelines are:

(1) Prefer raw, real-unit baseball metrics (mph, degrees, %, rpm) over min-max/normalized/embedding values; if only scaled values are available, clearly label them as non-interpretable for end users and ideally provide the underlying raw values too.
(2) Always include sample sizes (e.g., pitch count, PA count) when reporting tendencies/splits in specific situations or counts, and explicitly flag when the sample is too small to trust.
(3) When listing players, return human-readable player names (use name lookup) rather than only player IDs; IDs can be supplemental.
(4) If an output metric looks implausible for MLB context (e.g., team EV too low, fastball velocity too low, breaking-ball usage suspiciously low), the response should sanity-check, explain potential data/source limitations, or avoid making a definitive claim.
(5) When using category terms (e.g., "breaking balls", "fastballs"), define exactly which pitch types are included in that grouping to avoid ambiguity.
(6) For matchup-history questions with no direct data, do not stop at "no matchups"; provide alternative actionable analysis (pitcher approach vs. that handedness/profile, comps, or how the hitter matches up vs. the pitcher's main pitches).

Table 2: Compact artifact-to-claim map for reviewer verification.

Paper claim	Primary artifact(s)
Evaluation dataset creation and SME labeling session	`notebooks/04-Evaluation.ipynb` (FMAPI dataset generation, trace tagging, Review App labeling session creation)
Judge alignment from SME feedback via MemAlign	`notebooks/05-JudgeAlignment.ipynb` (MemAlign alignment, semantic/episodic memory inspection, aligned judge update)
Five-run GEPA prompt optimization design (100-question pool, disjoint 20-example subsets, 100 scorer calls/run)	`notebooks/06-PromptOptimization.ipynb`
Skill synthesis from tools, optimized prompt, aligned-judge artifacts, and traces	`notebooks/07-AgentSkillsGeneration.ipynb`
Runtime skill integration (metadata in prompt, on-demand `load_skill`)	`notebooks/08_create_agent_with_skills.ipynb`
Held-out comparison isolating skills effect under identical optimized prompt	`notebooks/09-Evaluation.ipynb`
UC-first routing, sufficiency node, Genie fallback, and parallel tool execution	`notebooks/03_create_agent_definition.ipynb`

(7) For pitch-arsenal questions, include both the list of pitch types and their usage frequencies (overall and, when relevant, by handedness/count).

At inference time, MemAlign constructs a working memory from all seven guidelines plus the most relevant retrieved examples, enabling the judge to score new traces informed by accumulated expert knowledge.

D System Prompt Key GEPA Additions

The following sections were added or substantially rewritten by GEPA during prompt optimization. They were absent from the original prompt entirely.

D.1 Embedding Data Handling

Listing 2: GEPA-added embedding handling rules.

```
CRITICAL: Handling Embedding Data

The output from embedding vectors is minmax scaled (0.0 to
    1.0).
- NEVER output these raw float values (e.g., "0.745 speed")
    in your response.
- Interpret as percentiles or qualitative descriptors
    relative to the league (0.90 = "Elite/Top-tier", 0.50 =
     "League Average", 0.20 = "Below Average").
- ALWAYS prefer raw units (MPH, RPM, Inches of break) from
    other tools over these scaled values.
- Sanity Check: If a metric seems implausible for its
    standard unit (e.g., a spin rate of 0.645), assume it
    is normalized. Do not present it as a raw value.

If raw data is unavailable, explain the limitation clearly
    rather than presenting confusing numbers.
```

D.2 Tool Parameter Rules

Listing 3: GEPA-added tool parameter rules.

```
- get_pitcher_tendency_by_count: b (0-3) and s (0-2) are
    REQUIRED integers. If asking for general tendencies,
    query representative counts (0-0, 1-1, 0-2, 3-1).
- get_pitcher_tendency_with_runners: b, s are REQUIRED
    integers. p_on_1b, p_on_2b, p_on_3b are REQUIRED
    booleans (true/false). Never pass null.
- lookup_player_by_name: Always call this first to get IDs.
```

D.3 Sample Size Requirements

Listing 4: GEPA-added sample-size rules.

```
- When reporting tendencies (especially by count), MUST
    explicitly state the sample size (e.g., "Based on 49
    pitches" or "In 150 plate appearances"). Without this,
    the reliability of the data is unknown.
- Sample Size Warnings: Conclusions from small samples (< 5
    ABs) should be treated with caution.
- If user requests N at-bats but fewer are returned, present
     available data then MANDATORY pivot to broader
    analysis to fill the gap. Do not stop at limited data.
```

These additions trace directly to the aligned judge's learned guidelines: guideline 1 produced the embedding-handling rules; guideline 2 produced the sample-size requirements; guidelines 6 and 7 produced the matchup-history and arsenal instructions. GEPA translated evaluation criteria into operational prompt instructions.

E What Changed Due to Labeling

Table 3 summarizes how one labeling session is transformed into concrete system changes by the bridge.

F Before/After Agent Response

Query: "How will Kyle Freeland pitch to Freddie Freeman?"

Table 3: Label-to-change traceability: expert feedback to aligned judge memory to automated prompt/skill updates to behavior changes.

Label feedback pattern	Aligned-judge guideline	Automated system change	Observed behavior change
Responses reported normalized values as if they were raw baseball units	Prefer real-unit metrics; flag normalized values	Prompt adds explicit embedding handling rules and unit sanity checks (Appendix D)	Responses avoid presenting minmax values as mph/rpm and explain limits when raw units are unavailable
Tendency statements omitted reliability context	Always include sample sizes and small-sample warnings	Prompt adds required sample-size reporting and caution rules (Appendix D)	Count-based recommendations now include pitch/PA counts and reliability caveats
No direct matchup data led to dead-end answers	Provide alternative actionable analysis when direct history is absent	Prompt and skills add fallback strategy patterns for no-history cases (Appendices D, G)	Responses pivot from "no data" to actionable plan using tendencies/comps
Pitch repertoire summaries were incomplete or underspecified	Include both pitch list and usage frequencies; define category groupings	Prompt and generated skills enforce arsenal completeness and explicit categorization (Appendices D, G)	Reports include full arsenal with usage context and clearer tactical recommendations

Pre-optimization response. The original agent response is directionally useful but less structured and less explicit about count-specific usage, reliability context, and full arsenal detail than the updated response.

Listing 5: Pre-optimization response excerpt.

```
### Data Collected
Analyzed 24 pitches from Freeman-Freeland matchups in 2025
     (2 doubles, 2 strikeouts, 1 field out) and Freeland's
     tendencies vs LHB across key counts (0-0, 1-0, 0-2,
     2-2, 3-2).

### Pitcher Approach
- First Pitch (0-0): Freeland establishes with his four-seam
      fastball and sweeper.
- Ahead (0-2): Freeland becomes breaking-ball oriented with
     knuckle curve and sweeper, targeting away/low.
- Hitter's counts (1-0, 2-2, 3-2): leans fastball/sweeper
     mix with count-dependent locations.

Notable: Freeman has hit Freeland hard (104.6 MPH and 89.6
     MPH EV) including a double on a 2-2 middle-in fastball.

### Recommendation
Be aggressive early on fastballs up in the zone. In two-
     strike counts, lay off the knuckle curve low-away. In
     hitter's counts, sit fastball middle-away and avoid
     chasing sweepers off the plate.
```

Post-optimization response. The updated response includes full arsenal detail with velocity bands, per-count usage with explicit sample sizes, and head-to-head evidence tied to count-specific recommendations.

Listing 6: Post-optimization response excerpt.

```
### Data Collected
Freeland's 2025 pitch arsenal (6 pitch types), count
     tendencies vs LHB (0-0, 1-1, 0-2), and direct H2H
     history vs Freeman (25 pitches across 4 PA).

### Pitcher Approach
Arsenal Overview (vs LHB):
- 4-Seam Fastball: 90-93 MPH, primary weapon
- Sweeper: 81-84 MPH, key breaking ball
- Knuckle Curve: 82-84 MPH, putaway pitch
- Cutter: occasional usage
- Sinker/Changeup: minimal vs LHB

0-0 (N=147): fastballs 59% (4-seam 35%, cutter 17%),
     breaking balls 24%.
1-1 (N=48): breaking balls 52% (KC 25%, ST 27%).
0-2 (N=41): breaking-ball dependent 71% (KC 41%, ST 29%).

Head-to-Head (2025, N=25 pitches):
- Sweeper 32% (N=8), one hard-hit ball (104.6 MPH EV)
- 4-Seam 40% (N=10), one double (89.6 MPH EV)
- Knuckle Curve 24% (N=6), two strikeouts

### Recommendation
Hunt the fastball early (0-0, 1-0). Protect low-away with
     two strikes: Freeland's putaway mix is 71% breaking
     balls (KC 41%, ST 29%). Be ready to punish middle-in
     fastballs and lay off breaking balls below the knees
     away.
```

Key differences reflected in the canonical response file are: explicit count-by-count sample sizes, expanded arsenal coverage with velocity bands, clearer pitch-usage distributions per count, and recommendations directly tied to both tendency data and head-to-head evidence. These additions correspond to the aligned judge's learned guidelines and the GEPA-produced prompt rules in Appendix D.

G Generated Agent Skills via optimize_anything

Section 3.4 describes how GEPA's `optimize_anything` iteratively refines skill text. The optimizer produced seven multi-file skills,

each decomposed into `skill.md`, `gotcha.md`, and `examples.md`, all without manual authoring:

(1) `situational-pitching-analysis`
(2) `pitcher-scouting-report`
(3) `h2h-matchups`
(4) `similar-player-finder`
(5) `roster-strategy`
(6) `lineup-optimization`
(7) `league-analysis-genie`

Below is an excerpted example for the situational-pitching domain.

Listing 7: GEPA-generated situational-pitching skill excerpt.

```
# --- skill.md ---
---
name: situational-pitching-analysis
description: >
  Use this skill for queries about specific game states
  involving runners on base. Triggered by "runners in
  scoring position", "runner on 1st", "bases loaded",
  or "how does he pitch from the stretch?".
---

## Tools
- lookup_player_by_name: Resolves pitcher ID.
- get_pitcher_tendency_with_runners: Primary tool for runner
    -state queries.
- parallel_tools: Required for aggregating RISP states.

## Workflow
1. Identify Pitcher: Call lookup_player_by_name.
2. Define Scenario & Execute:
   - Specific Base (e.g., "Runner on 1st"): Call
     get_pitcher_tendency_with_runners with exact flags (
     p_on_1b=True, others False).
   - RISP: NOT a single flag. MUST use parallel_tools to
     query THREE states (2nd only, 3rd only, 2nd & 3rd) and
     aggregate.
   - Bases Loaded: Set all flags to True.
3. Synthesize: Sum pitch counts from parallel results for "
    Total RISP". Do NOT average the averages.
4. Fallback: If N < 15, prepend warning: "Caution: Very
    small sample size (N=XX)."

## Quality expectations
- Boolean flags must strictly match the user's request.
- Always provide MPH/RPM.
- MUST include raw pitch counts for every percentage.

# --- gotcha.md ---
1. NEVER ignore count parameters. If tool requires b and s
    and user didn't specify, assume 0-0 or run multiple
    queries. Do not pass null.
2. CRITICAL: "RISP" is a concept, not a tool parameter. MUST
     query Runner on 2nd AND Runner on 3rd separately and
    aggregate. p_on_2b=True alone is incomplete.
3. DO NOT hallucinate velocity drops. Only state decrease if
     data explicitly shows lower release_speed.
4. MUST handle zero results. State "No data available for
    this specific base state in 2025."

# --- examples.md ---
### Example 1: Runner on First
User: "How does Glasnow pitch with a runner on first?"
Tool Sequence:
1. lookup_player_by_name -> ID 607192
2. parallel_tools:
   - get_pitcher_tendency_with_runners(p_id=607192, b_hand='
     R', b=0, s=0, p_on_1b=True, p_on_2b=False, p_on_3b=
     False)
   - Same for b_hand='L'
Response: "Tyler Glasnow with Runner on 1st: Four-Seam
    Fastball: 60% usage (N=85). Velocity holds at 97 MPH.
    Curveball: Usage drops to 20% (N=28). Expect the heater
    ."

### Example 2: Bases Loaded
User: "What does Hader throw with the bases loaded?"
Tool Sequence:
1. lookup_player_by_name -> ID 623352
2. get_pitcher_tendency_with_runners(p_id=623352, b_hand='R
    ', b=0, s=0, p_on_1b=True, p_on_2b=True, p_on_3b=True)
Response: "Josh Hader - Bases Loaded (2025). Caution: Small
    sample (N=12 pitches). Sinker: 83% usage (10/12).
    Slider: 17% (2/12). He is challenging you with the
    fastball."
```

The iterative refinement against a composite scorer produces skills with conditional branching in the workflow, explicit data-gap handling in gotchas, and concrete worked examples with tool sequences and sample responses. Quality expectations trace directly to the aligned judge's guidelines (Appendix C).

H Held-Out Evaluation Results

Figure 2 visualizes the Phase B held-out scores from Table 1. All three configurations use the same aligned judge and the same 30-question held-out set unseen during optimization. The baseline and optimized-prompt agents share the same `agent.py`; the only difference is the system prompt. The optimized-prompt and optimized-prompt+skills agents share the same GEPA-optimized prompt; the only difference is the presence of generated skills. Error bars show standard deviation across the 30 questions.

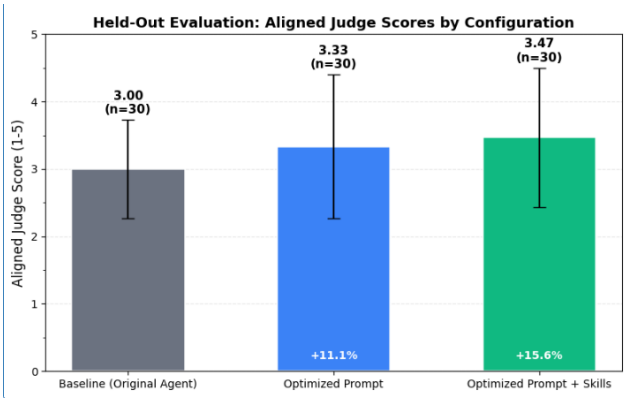


Figure 2: Held-out aligned-judge scores (1 to 5) by agent configuration. Baseline: original agent with optimized prompt (3.00). Optimized Prompt: GEPA-optimized prompt only (3.33, +11.0%). Optimized Prompt + Skills: GEPA-optimized prompt with generated agent skills (3.47, +15.7%). All evaluated on the same 30-question held-out set with the same aligned judge.